
Formalizing Group Action in Lean

Student name: *Frankie Feng-Cheng WANG*

Email: *maths@frankie.wang*

GitHub: *<https://github.com/FrankieeW/GroupAction>*

Course: *MATH70040-Formalising Mathematics* – Lecturer: *Dr Bhavik Mehta*

Due date: *January 30, 2026*

Contents

1	Introduction	2
1.1	Running the Code	2
2	Definitions	3
2.1	Group Action	3
2.2	Faithful and transitive actions	4
3	Concrete Examples of Group Actions	4
3.1	Symmetric group acting on a set	4
3.2	Group acting on itself by left multiplication	5
3.3	Subgroup acting on the group	5
3.4	Conjugation action of a subgroup on itself	6
3.5	Scalar action on complex vector spaces	6
3.6	Dihedral group action on a square	7
4	Permutation Representation	7
4.1	Proof Path	7
4.2	Step-by-Step Proof with Mathematical and Code Explanations	8
	Step 1: Define the action map σ_g	8
	Step 2: Prove σ_g is a bijection	8
	Step 3: Construct the permutation representation ϕ	9
	Step 4: Verify homomorphism properties	9
	Step 5: Package the theorem	10
4.3	Proof sketch (informal)	11
4.4	Lean code	11
5	Stabilizers	12
5.1	Proof sketch (informal)	13
5.2	Lean code	13
6	What Lean Guarantees	14

7	Reflection	14
7.1	Specific challenges encountered about formalization	14
7.2	Specific challenges encountered about code structure and style	15
8	Conclusion	15
8.1	Code Quality Verification	15
9	Future Extensions	16
10	References	16
I	Appendix	16
A	AI Assistance Statement	16
B	Lean Tactics Glossary	16
C	Project Configuration and Build Instructions	17
3.1	Project Structure	17
3.2	Lake Configuration	17
3.3	Dependencies	18

1. Introduction

This report presents a Lean 4 formalisation of **group actions**. The development is organised around three closely related components:

1. **Core definitions:** the `GroupAction` typeclass, together with the notions of faithfulness and transitivity;
2. **Concrete examples:** a selection of standard group actions illustrating the abstract definitions;
3. **Key theorems:** the construction of the permutation representation associated to a group action and the subgroup structure of stabilisers.

The mathematical development and theorem numbering follow Fraleigh and Katz, *A First Course in Abstract Algebra*, Addison-Wesley, 2003, Section 16 (Group Actions).

The complete Lean source code is available on GitHub:

`https://github.com/FrankieeW/GroupAction`

1.1. Running the Code. To verify the formalization, use Lake to build and run the Lean code:

```
lake build                # Build the project
lake env lean lean/GroupAction.lean    # Run the main file
lake env lean lean/GroupAction/Examples.lean  # Check all examples
```

All code snippets appearing in this report are hyperlinked to the corresponding lines in the repository.

The principal results formalised in this project are stated below and proved in later sections.

Theorem 16.3. Let X be a G -set. For each $g \in G$, the map

$$\sigma_g : X \rightarrow X, \quad \sigma_g(x) = g \cdot x,$$

is a permutation of X . Moreover, the assignment

$$\phi : G \rightarrow \text{Sym}(X), \quad \phi(g) = \sigma_g,$$

is a group homomorphism, and for all $g \in G$ and $x \in X$,

$$\phi(g)(x) = g \cdot x.$$

Theorem 16.12. Let X be a G -set and let $x \in X$. The stabiliser of x in G , defined by

$$G_x = \{ g \in G \mid g \cdot x = x \},$$

is a subgroup of G .

2. Definitions

This section introduces the group action class and the definitions of faithful and transitive actions.

2.1. Group Action. Now, we present the definition of a group action both mathematically and in Lean.

Mathematical definition.

A **group action** of a group G on a set X is a map

$$\cdot : G \times X \rightarrow X,$$

satisfying, for all $g_1, g_2 \in G$ and $x \in X$:

- (Compatibility) $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x)$;
- (Identity) $1 \cdot x = x$.

Lean formalisation.

The following Lean code defines a (left) action of a monoid G on a type X .

- `act : G → X → X` represents the action $g \cdot x$;
- `ga_mul` encodes compatibility with multiplication;
- `ga_one` states that the identity acts trivially.

```

13  /-- Defines a group action of a monoid `G` on a type `X`.
14  The action is given by `act : G → X → X`,
15  satisfying the axioms `ga_mul` and `ga_one`. -/
16  class GroupAction (G : Type*) [Monoid G] (X : Type*) where
17    /-- The action function that applies a group element to an element of
    ↪ `X`. -/

```

```

18  act : G → X → X
19  /-- The multiplication axiom: `(g₁ * g₂) • x = g₁ • (g₂ • x)` -/
20  ga_mul : ∀ g₁ g₂ x, act (g₁ * g₂) x = act g₁ (act g₂ x)
21  /-- The identity axiom: `1 • x = x` -/

```

Using Monoid instead of Group allows the action to be defined for monoids in general. The inverse element is not needed for the axioms. However, I will only consider group actions in this project.

```

13  variable {G : Type*} [Group G] {X : Type*} [GroupAction G X]

```

2.2. Faithful and transitive actions.

Faithful actions. An action of a group G on a set X is said to be **faithful** if distinct group elements induce distinct permutations of X . Equivalently, the action is faithful if

$$(\forall x \in X, g_1 \cdot x = g_2 \cdot x) \implies g_1 = g_2.$$

In other words, no nontrivial group element acts trivially on all of X .

```

25  /-! ## Definitions: act faithfully -/
26  /-- A group action is faithful if different group elements

```

Transitive actions. An action of a group G on a set X is **transitive** if for any $x_1, x_2 \in X$, there exists an element $g \in G$ such that

$$g \cdot x_1 = x_2.$$

This means that X consists of a single orbit under the action of G .

```

33  /-! ## Definitions: transitive -/
34  /-- A group action is transitive if for any `x₁, x₂ ∈ X`,

```

3. Concrete Examples of Group Actions

This section presents several concrete instances of the GroupAction class, illustrating how the abstract definition applies to familiar mathematical structures.

3.1. Symmetric group acting on a set. Let X be any type. The symmetric group $\text{Sym}(X)$ acts on X by evaluation: for $\sigma \in \text{Sym}(X)$ and $x \in X$, define $\sigma \cdot x := \sigma(x)$.

```

21  instance permGroupAction (X : Type*) : GroupAction (Equiv.Perm X) X :=
22  { act := fun g x => g x
23    ga_mul := by
24      intro g1 g2 x
25      rfl
26    ga_one := by
27      intro x
28      rfl }

```

The axiom proofs are immediate by reflexivity: composition of permutations and function evaluation commute definitionally in Lean.

Two standard properties of this action are faithful and transitive:

```

30 theorem permGroupAction_faithful (X : Type*) :
31   GroupAction.faithful (G := Equiv.Perm X) (X := X) :=
32 by
33   intro g₁ g₂ h
34   ext x
35   exact h x
36
37 theorem permGroupAction_transitive
38   (X : Type*) [DecidableEq X] :
39   GroupAction.transitive (G := Equiv.Perm X) (X := X) :=
40 by
41   intro x₁ x₂
42   refine ⟨Equiv.swap x₁ x₂, ?_⟩
43   simp [GroupAction.act]

```

The instance `DecidableEq X` is required because `Equiv.swap` constructs a permutation by branching on whether two elements are equal, which needs decidable equality.

3.2. Group acting on itself by left multiplication. Every group G acts on itself by left multiplication: $g_1 \cdot g_2 := g_1 * g_2$.

```

46 /-- A group acts on itself by left multiplication. -/
47 instance groupAsGSet (G : Type*) [Group G] : GroupAction G G :=
48   { act := fun g1 g2 => g1 * g2
49     ga_mul := by
50       intro g1 g2 g3
51       rw [mul_assoc]
52     ga_one := by
53       intro g
54       rw [one_mul] }

```

The `ga_mul` axiom follows from associativity of group multiplication, and `ga_one` from the identity axiom.

3.3. Subgroup acting on the group. A subgroup $H \leq G$ acts on G by left multiplication. Elements of H are coerced to elements of G using $(h : G)$.

```

58 /-- A subgroup acts on the ambient group by left multiplication. -/
59 instance subgroupAsGSet {G : Type*} [Group G] (H : Subgroup G) :
60   ↪ GroupAction H G :=
61   { act := fun h g => (h : G) * g
62     ga_mul := by
63       intros
64       simp [mul_assoc]
65     ga_one := by
66       intros
67       simp }

```

The `simp` tactic handles the coercion and applies associativity and identity axioms.

3.4. Conjugation action of a subgroup on itself. A subgroup H acts on itself by conjugation: $h_1 \cdot h_2 := h_1 h_2 h_1^{-1}$.

```

68 /-- A subgroup acts on itself by conjugation. -/
69 instance subgroupAsGSetConjugation {G : Type*} [Group G] (H : Subgroup G)
   ↪ : GroupAction H H :=
70   { act := fun h g => h * g * h⁻¹
71     ga_mul := by
72       intro g₁ g₂ g₃
73       simp
74       rw [← mul_assoc g₁]
75       rw [mul_assoc g₁ g₂]
76       rw [← mul_assoc]
77     ga_one := by
78       intros
79       simp }

```

The `mul_assoc` lemma is from `mathlib` and states associativity of multiplication:

```

1 mul_assoc.{u_1} {G : Type u_1} [Semigroup G] (a b c : G) :
2 a * b * c = a * (b * c)

```

The `ga_mul` proof can easily be done by `simp [mul_assoc]`, but I expanded it to show the steps explicitly by using `rw`.

For example the `rw [← mul_assoc g₁]` will automatically find the passible situation looks like $g_1 * (b * c)$, because we only give one argument g_1 to `← mul_assoc`. In detail, it will rewrite the left hand side $g_1 * (g_2 * g_3 * g_2^{-1})$ to $g_1 * (g_2 * g_3) * g_2^{-1}$ according to the right to left direction of the lemma `mul_assoc`.

3.5. Scalar action on complex vector spaces. The multiplicative group $\mathbb{C}^\times$ of nonzero complex numbers acts on $\mathbb{C}^n$ by scalar multiplication.

```

83 /-- The multiplicative group of complex numbers acts on complex vectors by
   ↪ scalar multiplication. -/
84 instance vectorSpaceAsCStarSet (n : ℕ) : GroupAction (ℂ*) (Fin n → ℂ) :=
85   { act := fun r v => fun i => (r : ℂ) * v i
86     ga_mul := by
87       intros r1 r2 v
88       ext i
89       simp [mul_assoc]
90     ga_one := by
91       intro v
92       ext i
93       simp }
94
95 #check vectorSpaceAsCStarSet 3

```

Here $\mathbb{C}^\times$ denotes the units of $\mathbb{C}$ (invertible elements), and $\text{Fin } n \rightarrow \mathbb{C}$ represents functions from $\{0, \dots, n-1\}$ to $\mathbb{C}$ (i.e., n -dimensional complex vectors). The `ext` tactic proves function equality pointwise.

3.6. Dihedral group action on a square. Let D_4 be the symmetry group of a square, acting on $\mathbb{Z}/4\mathbb{Z}$.

```

130  /-- The dihedral group  $D_4$  of order 8, symmetry group of a square. -/
131  abbrev D4 := DihedralGroup 4
132
133  /-- Type alias for vertices of a square, indexed by  $\mathbb{Z}\text{Mod } 4$ . -/
134  abbrev Vertex :=  $\mathbb{Z}\text{Mod } 4$ 
135
136  /-- Type alias for sides of a square, indexed by  $\mathbb{Z}\text{Mod } 4$ . -/
137  abbrev Side :=  $\mathbb{Z}\text{Mod } 4$ 
138
139  /-- Type alias for midpoints of sides of a square, indexed by  $\mathbb{Z}\text{Mod } 4$ . -/
140  abbrev Midpoint :=  $\mathbb{Z}\text{Mod } 4$ 
141
142  /-- The action of  $D_4$  on  $\mathbb{Z}\text{Mod } 4$  representing vertices/sides/midpoints of a
    ↪ square. -/
143  def d4Act (g : D4) (x :  $\mathbb{Z}\text{Mod } 4$ ) :  $\mathbb{Z}\text{Mod } 4$  :=
144    match g with
145    -- Rotation  $r_i$ :  $x \mapsto x - i$ 
146    | DihedralGroup.r i => x - i
147    -- Reflection  $s_i$ :  $x \mapsto i - x$ 
148    | DihedralGroup.sr i => i - x
149
150  /-- The dihedral group  $D_4$  acts on  $\mathbb{Z}\text{Mod } 4$  (vertices of a square). -/
151  instance d4ActionZMod4 : GroupAction D4 ( $\mathbb{Z}\text{Mod } 4$ ) :=
152    { act := d4Act
153      ga_mul := by
154        intro g1 g2 x
155        cases g1 <|> cases g2 <|>
156        simp [d4Act, sub_eq_add_neg] <|> ring
157      ga_one := by
158        intro x
159        simp [DihedralGroup.one_def, d4Act, -DihedralGroup.r_zero]
160    }
161
162  /-- The dihedral group  $D_4$  acts trivially on the center point. -/
163  instance d4CenterAction : GroupAction D4 PUnit :=
164    { act := fun _ => PUnit.unit
165      ga_mul := by
166        intros
167        rfl
168      ga_one := by
169        intros
170        rfl }

```

4. Permutation Representation

The core construction is the map $\sigma_g : X \rightarrow X$ given by $\sigma_g(x) = g \cdot x$. The inverse of σ_g is $\sigma_{g^{-1}}$, so σ_g is a permutation. This yields the permutation representation $\phi : G \rightarrow \text{Sym}(X)$.

4.1. Proof Path. To establish the permutation representation, we follow these steps:

1. **Define the action map:** For each $g \in G$, construct $\sigma_g : X \rightarrow X$ by $\sigma_g(x) = g \cdot x$.

2. **Prove σ_g is a bijection:** Show that σ_g has an inverse, namely $\sigma_{g^{-1}}$.
3. **Construct the permutation:** Package σ_g as an element of $\text{Sym}(X)$ using the inverse.
4. **Define the representation:** Set $\phi(g) = \sigma_g$ to get $\phi : G \rightarrow \text{Sym}(X)$.
5. **Verify homomorphism properties:** Prove that ϕ preserves multiplication and the identity.

4.2. Step-by-Step Proof with Mathematical and Code Explanations. We now walk through each step in detail, providing both mathematical reasoning and the corresponding Lean code.

Step 1: Define the action map σ_g . This step may not be strictly necessary in Lean code, but we include it to align with traditional mathematical proof structure, showing how the action map is constructed before proving its properties.

Mathematical explanation.

For each group element $g \in G$, we define a function $\sigma_g : X \rightarrow X$ that applies the group action: $\sigma_g(x) = g \cdot x$. This is simply the action function partially applied to g .

Lean code explanation.

The function `sigma` implements this definition. It takes a group element `g` and returns a function from `X` to `X` that applies the action.

```

27  /-- Defines the action map `sigma_g : X -> X` by `x -> g * x`. -/
28  def sigma (g : G) : X -> X :=
29    fun x => GroupAction.act g x
30  #check sigma

```

Step 2: Prove σ_g is a bijection. Lean requires an explicit construction of the inverse function to show that σ_g is a permutation.

Mathematical explanation.

To show σ_g is bijective, we need both injectivity and surjectivity. The key insight is that the inverse operation g^{-1} provides the inverse function: $\sigma_{g^{-1}} \circ \sigma_g = \text{id}_X$ and $\sigma_g \circ \sigma_{g^{-1}} = \text{id}_X$. This follows from the group axioms and the action compatibility.

Lean code explanation.

The `sigmaPerm` definition constructs a permutation by providing both the forward function (`toFun := sigma g`) and its inverse (`invFun := sigma g-1`). The proofs of `left_inv` and `right_inv` verify the two-sided inverse properties using the action axioms.

```

32  /-- Defines the permutation of `X` induced by `g`.
33    The inverse is given by the action of `g-1`. -/
34  def sigmaPerm (g : G) : Equiv.Perm X :=
35    { toFun := sigma g
36      invFun := sigma g-1
37      left_inv := by
38        intro x
39        -- Step 1: combine `g-1` and `g` using the action axiom.

```

```

40   calc
41     GroupAction.act g-1 (GroupAction.act g x) =
42       GroupAction.act (g-1 * g) x := by
43       simp using (GroupAction.ga_mul g-1 g x).symm
44       -- Step 2: simplify `g-1 * g` to `1`.
45       _ = GroupAction.act (1 : G) x := by
46       -- Alternative: use `congrArg` with `inv_mul_cancel g`.
47       -- congrArg (fun t => GroupAction.act t x) (inv_mul_cancel g)
48       simp
49       -- Step 3: the identity acts as the identity on `X`.
50       _ = x := GroupAction.ga_one x
51   right_inv := by
52     intro x
53     -- Step 1: combine `g` and `g-1` using the action axiom.
54   calc
55     GroupAction.act g (GroupAction.act g-1 x) =
56       GroupAction.act (g * g-1) x := by
57       simp using (GroupAction.ga_mul g g-1 x).symm
58       -- Step 2: simplify `g * g-1` to `1`.
59       _ = GroupAction.act (1 : G) x := by
60       -- Alternative: use `congrArg` with `mul_inv_cancel g`.
61       -- congrArg (fun t => GroupAction.act t x) (mul_inv_cancel g)
62       simp
63       -- Step 3: the identity acts as the identity on `X`.
64       _ = x := GroupAction.ga_one x }
65 #check sigmaPerm

```

Step 3: Construct the permutation representation ϕ . This step bridges the individual permutations σ_g to a group homomorphism, constructing the representation map $\phi : G \rightarrow \text{Sym}(X)$ that preserves the group structure.

Mathematical explanation.

Having established that each σ_g is a permutation, we can define the representation map $\phi : G \rightarrow \text{Sym}(X)$ by $\phi(g) = \sigma_g$. This assignment preserves the action: $\phi(g)(x) = \sigma_g(x) = g \cdot x$.

Lean code explanation.

The function `phi` simply wraps `sigmaPerm`, and `phi_apply` confirms that it agrees with the original action.

```

67 /-- Defines the permutation representation `phi : G → Equiv.Perm X`.
68   This sends `g` to the permutation `σ_g`. -/
69 def phi (g : G) : Equiv.Perm X :=
70   sigmaPerm g
71
72 /-- `phi` agrees with the action on each element of `X`. -/
73 lemma phi_apply (g : G) (x : X) : phi g x = GroupAction.act g x := rfl

```

Step 4: Verify homomorphism properties. This step establishes that ϕ is a group homomorphism by verifying it preserves multiplication and the identity element, completing the proof that ϕ is a valid group representation.

Mathematical explanation.

For ϕ to be a group homomorphism, we need $\phi(g_1 g_2) = \phi(g_1) \phi(g_2)$ and $\phi(1) = 1$. The multiplication property follows from the action compatibility axiom: $\phi(g_1 g_2)(x) = (g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = \phi(g_1)(\phi(g_2)(x))$.

Lean code explanation.

The lemmas `phi_mul` and `phi_one` prove these properties. `phi_mul` uses `Equiv.ext` for extensionality and applies the `ga_mul` axiom, while `phi_one` uses the `ga_one` axiom.

```

79  /-- Shows that `phi` respects multiplication:
80    `phi (g_1 * g_2) = phi g_1 * phi g_2`. -/
81  lemma phi_mul (g_1 g_2 : G) :
82    phi (X := X) (g_1 * g_2) = phi (X := X) g_1 * phi (X := X) g_2 := by
83    -- Extensionality reduces equality of permutations to pointwise equality.
84    apply Equiv.ext
85    intro (x : X)
86    calc
87      -- Step 1: expand `phi` and the action definition.
88      phi (g_1 * g_2) x = GroupAction.act (g_1 * g_2) x := rfl
89      -- Step 2: use the action axiom to separate `g_1` and `g_2`.
90      _ = GroupAction.act g_1 (GroupAction.act g_2 x) := GroupAction.ga_mul g_1
91      _   ↪ g_2 x
92  /-- `phi 1` is the identity permutation. -/
93  lemma phi_one : phi (1 : G) = (1 : Equiv.Perm X) := by
94    apply Equiv.ext
95    intro (x : X)
96    calc
97      phi (1 : G) x = GroupAction.act (1 : G) x := rfl
98      _ = x := GroupAction.ga_one x
99      _ = (1 : Equiv.Perm X) x := by
100        simp [Equiv.Perm.one_apply]

```

Step 5: Package the theorem. Now just need to combine everything into the final theorem statement.

Mathematical explanation.

We now have all the ingredients: a homomorphism ϕ that satisfies the required properties. This completes the construction of the permutation representation.

Lean code explanation.

The `theorem group_action_to_perm_representation` packages everything together, using the previously proved lemmas to construct the existential witness.

```

102  /-- The action yields a permutation representation. -/
103  theorem group_action_to_perm_representation :
104    ∃ (ψ : G → Equiv.Perm X),
105      (∀ g x, ψ g x = GroupAction.act g x) ∧
106      (∀ g_1 g_2, ψ (g_1 * g_2) = ψ g_1 * ψ g_2) ∧
107      (ψ 1 = 1) := by
108    exact ⟨phi, ⟨phi_apply, ⟨phi_mul, phi_one⟩⟩⟩
109    --
110    -- Expanded proof (step-by-step):
111    -- refine ?phi, ?_ -- provide ψ = phi
112    -- refine ?phi_apply, ?_ -- provide ψ g x = g • x

```

```

113 -- refine [phi_mul, ?_] -- provide  $\varphi(g_1 * g_2) = \varphi g_1 * \varphi g_2$ 
114 -- exact phi_one -- provide  $\varphi 1 = 1$ 

```

4.3. Proof sketch (informal). To show that σ_g is a bijection, compute $\sigma_{g^{-1}}(\sigma_g(x)) = (g^{-1}g) \cdot x = 1 \cdot x = x$, and similarly $\sigma_g(\sigma_{g^{-1}}(x)) = x$. The representation is defined by $\phi(g) = \sigma_g$, and multiplicativity follows from the action axiom:

$$\phi(g_1 g_2)(x) = (g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = (\phi(g_1)\phi(g_2))(x).$$

4.4. Lean code. First, define the underlying action map:

```

26
27 /-- Defines the action map  $\sigma_g : X \rightarrow X$  by  $x \mapsto g \cdot x$ . -/
28 def sigma (g : G) : X → X :=
29   fun x => GroupAction.act g x

```

Then package it as a permutation using the inverse action:

```

34 def sigmaPerm (g : G) : Equiv.Perm X :=
35   { toFun := sigma g
36     invFun := sigma g⁻¹
37     left_inv := by
38       intro x
39       -- Step 1: combine  $g^{-1}$  and  $g$  using the action axiom.
40       calc
41         GroupAction.act g⁻¹ (GroupAction.act g x) =
42           GroupAction.act (g⁻¹ * g) x := by
43             simp using (GroupAction.ga_mul g⁻¹ g x).symm
44             -- Step 2: simplify  $g^{-1} * g$  to  $1$ .
45             _ = GroupAction.act (1 : G) x := by
46               -- Alternative: use  $\text{congrArg}$  with  $\text{inv\_mul\_cancel } g$ .
47               -- congrArg (fun t => GroupAction.act t x) (inv_mul_cancel g)
48               simp
49               -- Step 3: the identity acts as the identity on  $X$ .
50               _ = x := GroupAction.ga_one x
51     right_inv := by
52       intro x
53       -- Step 1: combine  $g$  and  $g^{-1}$  using the action axiom.
54       calc
55         GroupAction.act g (GroupAction.act g⁻¹ x) =
56           GroupAction.act (g * g⁻¹) x := by
57             simp using (GroupAction.ga_mul g g⁻¹ x).symm
58             -- Step 2: simplify  $g * g^{-1}$  to  $1$ .
59             _ = GroupAction.act (1 : G) x := by
60               -- Alternative: use  $\text{congrArg}$  with  $\text{mul\_inv\_cancel } g$ .
61               -- congrArg (fun t => GroupAction.act t x) (mul_inv_cancel g)
62               simp
63               -- Step 3: the identity acts as the identity on  $X$ .
64               _ = x := GroupAction.ga_one x }

```

Using `#check` on `sigmaPerm` confirms it has type

```

65 #check sigmaPerm

```

This confirms the map $g \mapsto \sigma_g$ is a well-defined function from G to $\text{Sym}(X)$. Define the representation and its basic properties:

```

69 def phi (g : G) : Equiv.Perm X :=
70   sigmaPerm g
71
72 /-- `phi` agrees with the action on each element of `X`. -/
73 lemma phi_apply (g : G) (x : X) : phi g x = GroupAction.act g x := rfl
74
75 -- /-- A commented-out attempt at proving multiplicativity of `phi`. -/
76 -- lemma phi_mul : ∀ (g₁ g₂ : G), phi (g₁ * g₂) = phi g₁ * phi g₂ := by
77 --   sorry
78
79 /-- Shows that `phi` respects multiplication:
80   `phi (g₁ * g₂) = phi g₁ * phi g₂`. -/
81 lemma phi_mul (g₁ g₂ : G) :
82   phi (X := X) (g₁ * g₂) = phi (X := X) g₁ * phi (X := X) g₂ := by
83   -- Extensionality reduces equality of permutations to pointwise equality.
84   apply Equiv.ext
85   intro (x : X)
86   calc
87     -- Step 1: expand `phi` and the action definition.
88     phi (g₁ * g₂) x = GroupAction.act (g₁ * g₂) x := rfl
89     -- Step 2: use the action axiom to separate `g₁` and `g₂`.
90     _ = GroupAction.act g₁ (GroupAction.act g₂ x) := GroupAction.ga_mul g₁
91     _   ↪ g₂ x
92
93 /-- `phi 1` is the identity permutation. -/
94 lemma phi_one : phi (1 : G) = (1 : Equiv.Perm X) := by
95   apply Equiv.ext
96   intro (x : X)
97   calc
98     phi (1 : G) x = GroupAction.act (1 : G) x := rfl
99     _ = x := GroupAction.ga_one x
100    _ = (1 : Equiv.Perm X) x := by
101      simp [Equiv.Perm.one_apply]

```

Finally, package the existence statement:

```

103 theorem group_action_to_perm_representation :
104   ∃ (ψ : G → Equiv.Perm X),
105     (∀ g x, ψ g x = GroupAction.act g x) ∧
106     (∀ g₁ g₂, ψ (g₁ * g₂) = ψ g₁ * ψ g₂) ∧
107     (ψ 1 = 1) := by
108   exact ⟨phi, ⟨phi_apply, ⟨phi_mul, phi_one⟩⟩⟩

```

5. Stabilizers

For a fixed $x \in X$, the stabilizer is the subset

$$G_x = \{g \in G \mid g \cdot x = x\}.$$

In Lean this is first defined as a set, then shown to be the carrier of a subgroup, and finally packaged as a Subgroup.

5.1. Proof sketch (informal). The identity element fixes every x , so $1 \in G_x$. If g_1 and g_2 fix x , then $(g_1 g_2) \cdot x = g_1 \cdot (g_2 \cdot x) = g_1 \cdot x = x$, so $g_1 g_2 \in G_x$. If g fixes x , then $g^{-1} \cdot x = g^{-1} \cdot (g \cdot x) = (g^{-1} g) \cdot x = x$, so $g^{-1} \in G_x$.

5.2. Lean code. Start from the set definition:

```

24   `G_x = { g ∈ G | g • x = x }`. -/
25 def stabilizerSet (x : X) : Set G :=

```

Package it as a subgroup by checking closure: Define the subgroup structure by providing proofs of the subgroup axioms:

- `carrier`: the underlying set is `stabilizerSet x`.
- `one_mem'`: the identity fixes every x by the `ga_one` axiom.
- `mul_mem'`: if g_1 and g_2 fix x , then so does $g_1 g_2$ by the `ga_mul` axiom.
- `inv_mem'`: if g fixes x , then so does g^{-1} by combining `ga_mul` and `ga_one`.

```

28 /-- The stabilizer `G_x` as a subgroup of `G`. -/
29 def stabilizer (x : X) : Subgroup G := by
30   exact
31   { carrier := stabilizerSet (G := G) (X := X) x
32     one_mem' := by
33       -- The identity fixes every x.
34       simp [stabilizerSet, GroupAction.ga_one x]
35     mul_mem' := by
36       intro g₁ g₂ hg₁ hg₂
37       calc
38         -- Closure under multiplication via the action axiom.
39         GroupAction.act (g₁ * g₂) x = GroupAction.act g₁
40         ↪ (GroupAction.act g₂ x) := by
41           simp using (GroupAction.ga_mul g₁ g₂ x)
42         _ = GroupAction.act g₁ x := by
43           rw [hg₂]
44         _ = x := hg₁
45     inv_mem' := by
46       intro g hg
47       calc
48         -- If g fixes x, then g⁻¹ fixes x as well.
49         GroupAction.act g⁻¹ x = GroupAction.act g⁻¹ (GroupAction.act g
50         ↪ x) := by
51           rw [hg]
52         _ = GroupAction.act (g⁻¹ * g) x := by
53           simp using (GroupAction.ga_mul g⁻¹ g x).symm
54         _ = GroupAction.act (1 : G) x := by simp

```

Then show the set is the carrier of a subgroup:

```

55 /-- The stabilizer set is the carrier of a subgroup of `G`. -/
56 theorem stabilizer_set_is_subgroup (x : X) :
57   ∃ H : Subgroup G, (H : Set G) = stabilizerSet (G := G) (X := X) x := by

```

6. What Lean Guarantees

Formalizing these constructions in Lean provides guarantees that informal proofs cannot match. When Lean accepts a definition or theorem, it has verified:

- **Types are correct.** The function `phi : G → Equiv.Perm X` has the claimed type. Lean checks that `sigmaPerm g` is actually a permutation (a bijection), not just a function. If we mistakenly defined a non-bijective map, Lean would reject it.
- **No hidden assumptions.** Every lemma explicitly lists its hypotheses. If a proof uses associativity, the code must invoke `mul_assoc` or the `ga_mul` axiom. There are no “clearly” or “it follows that” steps that skip justification.
- **Axioms match definitions.** The `GroupAction` class requires `ga_mul` and `ga_one`. If we omitted `ga_one`, the proof of `phi_one` would fail because Lean would have no way to show $1 \cdot x = x$.
- **Subgroup structure is verified.** When we package the stabilizer as a `Subgroup`, Lean checks that we provided proofs of closure under multiplication, inverses, and identity. The type system prevents us from claiming “the stabilizer is a subgroup” without proving it.

These checks prevent common errors: mistaken claims about injectivity, forgotten hypotheses, and gaps in subgroup proofs. An external examiner can trust that the Lean code genuinely establishes the mathematical claims, not just that it compiles.

7. Reflection

Writing the proofs in Lean forced me to make every step explicit. In particular:

- I had to show explicitly that $\sigma_{g^{-1}}$ is an inverse.
- I had to unfold the definition of permutation multiplication.

The resulting code is not shorter than a textbook proof, but it is more precise. An external examiner can read the surrounding explanations and then see that each Lean line matches a specific mathematical claim.

7.1. Specific challenges encountered about formalization. Several technical hurdles emerged during the formalization:

- **Typeclass resolution:** Ensuring Lean could automatically find the `GroupAction` instance in complex contexts required careful use of explicit type annotations (e.g., `(G := G)`).
- **Equivalence mechanics:** Constructing `sigmaPerm` as an `Equiv.Perm X` required providing both `left_inv` and `right_inv` proofs, which meant carefully applying the `ga_mul` axiom in both directions.
- **Coercions:** Working with subgroups required understanding how Lean coerces elements of type `H` (a subgroup) to type `G` (the ambient group) using `(h : G)`.

- **Function extensionality:** Proving equality of permutations required `Equiv.ext`, and equality of vector-valued functions required `ext i`, which were not immediately obvious.
- **Decidable equality:** Some examples (like the symmetric group action) required `DecidableEq` instances to construct certain permutations, which added complexity.
- **mul_assoc usage:** I initially attempted to add `mul_assoc` to the `simp` set because I used `simp [mul_assoc]` frequently. However, I later realized that `simp` does not include `mul_assoc` by design: changing the order of multiplication does not constitute a genuine simplification, since associativity merely rearranges the expression rather than making it simpler.

7.2. Specific challenges encountered about code structure and style. During the development, I faced several challenges related to code structure and style:

- **Modularization:** Deciding how to split the code into multiple files (e.g., `Defs.lean`, `Examples.lean`, `Permutation.lean`, `Stabilizer.lean`) required careful consideration of dependencies and logical flow.
- **Naming conventions:** Choosing clear and consistent names for definitions, lemmas, and instances was crucial for readability but sometimes challenging.
- **Documentation:** Balancing inline comments with external documentation to ensure that both the code and the report were understandable without redundancy.
- **Tactic selection:** Deciding when to use high-level tactics like `simp` versus more explicit steps with `rw` and `calc` for clarity.

8. Conclusion

This formalization establishes the foundational concepts of group actions in Lean, including the permutation representation and stabilizers.

8.1. Code Quality Verification. All Lean code in this project has been verified to pass `#lint` checks, ensuring high code quality and adherence to best practices:

- **Documentation completeness:** All definitions, theorems, and instances have proper documentation strings
- **No unused variables:** All declared variables are used in their respective contexts
- **No dead code:** All code paths are reachable and meaningful
- **Style consistency:** Code follows consistent naming conventions and formatting
- **Type safety:** No unsafe type coercions or suppressions

The project configuration in `lakefile.toml` enables comprehensive linter checks, and all files compile without warnings when running `lake build` with `linter.all = true`. This ensures the formalization is not only mathematically correct but also maintainable and follows community standards.

9. Future Extensions

This formalization provides a foundation for further development in group action theory. Potential extensions include: the orbit-stabilizer theorem relating the size of orbits to cosets of stabilizers; Burnside’s lemma for counting group elements with fixed points; group actions on cosets, leading to the concept of permutation representations; and applications to representation theory and algebraic topology.

10. References

John B. Fraleigh, Victor J. Katz, *A First Course in Abstract Algebra*, Addison-Wesley, 2003, Section 16 (Group Actions).

Part I

Appendix

A. AI Assistance Statement

I used AI-assisted tools in a limited and strictly auxiliary manner. Specifically, AI was used to help automate the conversion of Lean source files into $\text{\LaTeX}$ code blocks for inclusion in this report, and to assist with reorganising a single Lean file into multiple smaller files during the development process. In addition, AI was occasionally used for high-level brainstorming about structure and presentation. Additionally, C. Project Configuration and Build Instructions is generated with AI.

All mathematical content, Lean definitions, proofs, and formal statements were written, checked, and verified by me. AI tools were not used to replace mathematical reasoning or to generate unverified results. I take full responsibility for the correctness and originality of both the Lean code and the written exposition.

B. Lean Tactics Glossary

This section provides a brief explanation of the key Lean tactics used in the proofs throughout this formalization.

- **intro**: Introduces assumptions into the local context. For example, `intro g x` brings a group element g and a set element x into scope for a proof.
- **apply**: Applies a lemma or theorem to the current goal. For example, `apply ga_mul` uses the group action multiplication axiom.
- **simp**: Simplifies expressions using equational lemmas and basic rules. For example, `simp` might simplify $(g_1 g_2) \cdot x$ to $g_1 \cdot (g_2 \cdot x)$ using associativity.
- **rw**: Rewrites the goal or hypotheses using an equality. For example, `rw [ga_one]` rewrites using the identity axiom.
- **calc**: Structures multi-step equalities in a readable way. For example, `calc` blocks break down complex equality proofs into clear steps.

- **ext**: Proves equality of functions by proving they agree on all inputs. For example, `ext x` proves two functions are equal by showing they produce the same output for every x .
- **exact**: Closes a goal by providing an exact term that matches the goal type. For example, `exact rfl` proves $x = x$.
- **refine**: Partially constructs a term, leaving holes for subgoals. For example, `refine { act := fun g x => g x, ga_mul := ?_ }` constructs a `GroupAction` instance with a hole for the multiplication proof.
- **constructor**: Builds values of inductive types. For example, `constructor` might construct an `Equiv.Perm X` by providing the forward and inverse functions.

These tactics form the core toolkit for interactive theorem proving in Lean. Each tactic has precise semantics and helps build formal proofs step by step.

C. Project Configuration and Build Instructions

This section documents the project setup, configuration, and build process for reproducibility.

3.1. Project Structure. The project follows a modular organization:

```

GroupAction/
├─ lean/                                # Lean 4 source files
│   └─ GroupAction.lean                # Main entry point
│       └─ GroupAction/                # Core modules
│           ├── Defs.lean               # GroupAction class + axioms
│           ├── Basic.lean              # Basic lemmas (faithful, transitive)
│           ├── Permutation.lean        #  $\phi : G \rightarrow \text{Equiv.Perm } X$ 
│           ├── Stabilizer.lean         # Stabilizer subgroup  $G_x$ 
│           └─ Examples.lean            # Instances and #check tests
├─ tex/                                # LaTeX report + figures
├─ doc/                                # Project documentation
├─ lakefile.toml                       # Lake configuration
├─ lean-toolchain                      # Pinned Lean version
└─ README.md                          # Project overview

```

3.2. Lake Configuration. The project uses Lake (Lean’s build system) with the following key configuration in `lakefile.toml`:

```

[leanOptions]
pp.unicode.fun = true           # Enable Unicode function notation
autoImplicit = false            # Disable automatic implicit arguments
relaxedAutoImplicit = false     # Strict implicit argument handling
linter.style.longLine = true    # Enable long line linter

# Additional linter options (recommended for code quality)
linter.all = true               # Enable all recommended linters

```

```
linter.missingDocs = true      # Check for missing documentation
linter.unusedVariables = true  # Check for unused variables
linter.unusedArguments = true  # Check for unused arguments
linter.deadCode = true         # Check for dead code
```

3.3. Dependencies.

- **Lean 4:** Version 4.27.0 (pinned in `lean-toolchain`)
- **mathlib:** Version 4.27.0 (via Lake)
- **LaTeX:** XeLaTeX with minted package for code highlighting

This configuration ensures reproducible builds, consistent code quality, and clear documentation throughout the project lifecycle.